Centro Federal de Educação Tecnológica de Minas Gerais

Trabalho Prático 2 Sistemas Distribuídos

Implementação e avaliação de um sistema de transferência de arquivos Peer-to-Peer (P2P)

Gabriel Araujo Barbosa

Belo Horizonte
2026

1. Introdução

O objetivo deste trabalho foi implementar um sistema de transferência de arquivos utilizando o modelo Peer-to-Peer (P2P), no qual os próprios participantes da rede podem enviar e receber dados. Nesse modelo, cada peer atua tanto como cliente quanto como servidor, permitindo a distribuição dos arquivos sem depender de um servidor central para as transferências.

A implementação foi desenvolvida em Python, com sockets TCP e a biblioteca asyncio. O sistema permite fragmentar arquivos em blocos, compartilhar fragmentos entre vizinhos e reconstruir o arquivo ao final. Os experimentos foram realizados com diferentes quantidades de peers, tamanhos de blocos e arquivos, conforme o enunciado do Trabalho Prático 2.

2. Comunicação entre os peers utilizando sockets

Cada peer executa um servidor TCP para atender solicitações e, ao mesmo tempo, inicia conexões como cliente para obter dados de vizinhos. Os endereços IP e portas são configurados previamente, dispensando o uso de um tracker. A biblioteca asyncio possibilita atender conexões simultâneas e solicitar blocos sem bloquear a execução do servidor.

Nos testes com quatro peers, foi utilizada uma topologia em cadeia, P0 → P1 → P2 → P3. O peer P0 inicia com o arquivo completo e os demais o recebem gradualmente. Assim que um leecher recebe e valida um bloco, ele pode repassá-lo ao próximo peer, mesmo antes de terminar o próprio download. O protocolo inclui consultas de metadados, inventário de blocos disponíveis e pedidos de fragmentos específicos.

3. Fragmentação e remontagem de arquivos

O arquivo de origem é dividido logicamente em blocos de tamanho fixo, com 1.024 ou 4.096 bytes. Os metadados são armazenados em JSON e incluem o nome e o tamanho do arquivo, o tamanho do bloco, o SHA-256 global e o SHA-256 de cada fragmento. As mensagens de rede usam quatro bytes para o tamanho do cabeçalho JSON, seguidos do próprio cabeçalho e, quando necessário, dos dados binários.

Cada peer mantém um mapa de bits que informa quais blocos já possui. Os fragmentos são conferidos pelo tamanho e pelo hash antes de serem gravados na posição correta do arquivo parcial. Quando todos os blocos estão presentes, são verificados o tamanho e o SHA-256 do arquivo reconstruído. O programa também registra eventos da transferência e mantém informações locais para recuperar blocos válidos em uma execução posterior.

4. Implementação e testes

Os estudos de caso foram executados no meu computador, em Windows. Todos os peers foram iniciados como processos independentes, comunicando-se por TCP no endereço local 127.0.0.1. Foram utilizados dois ou quatro peers, blocos de 1.024 ou 4.096 bytes e arquivos gerados com conteúdo binário pseudorrandômico reproduzível.

Foram avaliados os arquivos A, com 10 e 20 KiB, B, com 1 e 5 MiB, e C, com 10 e 20 MiB. Além dos seis casos principais, quatro comparações adicionais mantiveram o tamanho do arquivo constante para avaliar separadamente o efeito da quantidade de peers e do tamanho dos blocos. O programa registrou o tempo total de cada cenário, o número de blocos, a origem dos fragmentos e a verificação de integridade.

5. Resultados

Cenário	Arquivo	Peers	Bloco (B)	Blocos	Tempo (s)
A - padrão	10 KiB	2	1.024	10	0,58
A - variação	20 KiB	4	4.096	5	0,62
B - padrão	1 MiB	2	1.024	1.024	16,05
B - variação	5 MiB	4	4.096	1.280	21,96
C - padrão	10 MiB	2	1.024	10.240	107,35
C - variação	20 MiB	4	4.096	5.120	84,11
B - 4 peers	1 MiB	4	1.024	1.024	29,35
B - bloco 4 KiB	1 MiB	2	4.096	256	4,58
C - 4 peers	10 MiB	4	1.024	10.240	168,67
C - bloco 4 KiB	10 MiB	2	4.096	2.560	26,18

Tabela 1 - Resultados dos dez cenários (uma execução por cenário).

Todos os dez cenários foram concluídos com status PASS. Em cada um deles, os downloads dos leechers apresentaram os tamanhos esperados e SHA-256 igual ao do arquivo de origem. Nos seis casos principais, os tempos totais variaram de 0,58 s, para o arquivo A de 10 KiB, a 107,35 s, para o arquivo C de 10 MiB com blocos de 1.024 bytes. O caso de 20 MiB com quatro peers e blocos de 4.096 bytes levou 84,11 s.

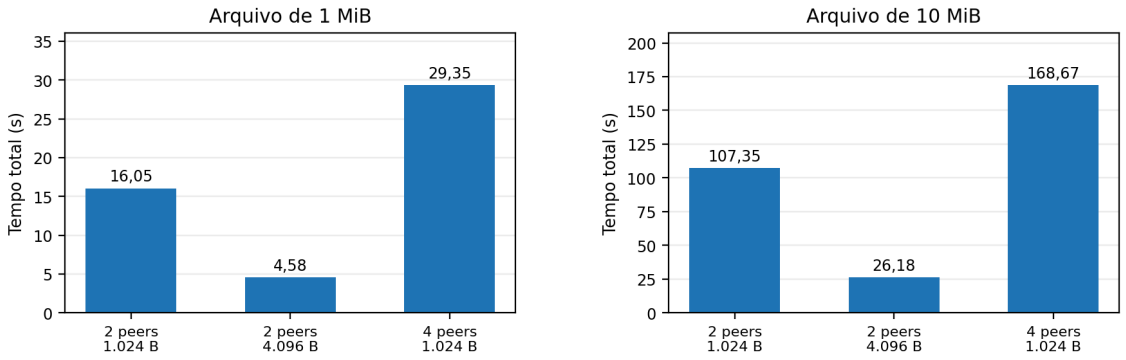


Figura 1 - Comparação de tempos totais com arquivos de 1 MiB e 10 MiB.

Com 1 MiB e dois peers, aumentar o bloco de 1.024 para 4.096 bytes reduziu o tempo de 16,05 s para 4,58 s. Com 10 MiB, a redução foi de 107,35 s para 26,18 s. Isso ocorreu junto com a redução da quantidade de blocos: de 10.240 para 2.560 no arquivo de 10 MiB, diminuindo o número de requisições necessárias.

Para 1 MiB com blocos de 1.024 bytes, utilizar quatro peers em cadeia elevou o tempo de 16,05 s para 29,35 s. Com 10 MiB, o tempo passou de 107,35 s para 168,67 s. Nesse arranjo, cada peer adicional precisa receber os dados do anterior. Portanto, aumentar o número de peers não garante uma transferência mais rápida.

Nos cenários com quatro peers, o registro da origem dos blocos confirmou que P2 obteve fragmentos de P1 e P3 de P2, demonstrando o compartilhamento entre peers intermediários. Os tempos apresentados correspondem a uma única execução por cenário e incluem inicialização, coordenação, transferência e verificação. Por serem medições locais, não representam médias nem uma avaliação de desempenho em redes remotas.

6. Conclusão

O trabalho permitiu observar o funcionamento de uma rede P2P simétrica, com cada processo exercendo simultaneamente as funções de cliente e servidor. A fragmentação, o gerenciamento dos blocos e a remontagem foram validados nos dez cenários, e a comparação dos hashes SHA-256 comprovou a integridade dos arquivos recebidos.

Os experimentos também mostraram que blocos maiores podem reduzir o tempo de transferência ao diminuir a quantidade de solicitações. Já o uso de mais peers em uma cadeia acrescentou etapas de comunicação e aumentou o tempo total nos casos comparáveis. Dessa forma, o desempenho depende não apenas da quantidade de participantes, mas também do tamanho dos blocos e da organização da rede.

Código-fonte: https://github.com/gabaraujodev/TP2-Sistemas-Distribuidos